

I 1 Nguồn giám sát ngày càng rẻ (Increasingly Cheap Supervision)

PHỤ THUỘC B/06-chat-luong-du-lieu (nhân đất và bắn tới mức nào); B/09-thiet-ke-ham-mat-mat (*contrastive loss, cross-entropy*); C/13-kien-truc-backbone/03-vit-va-inductive-bias (vì sao ViT cần nhiều dữ liệu hơn CNN); C/14-bai-toan-cv-loi (*open-vocabulary detection* là hệ quả trực tiếp của mục này).

TÓM TẮT

Mục này xếp bốn nguồn giám sát theo chi phí giảm dần: nhãn của người khác, chính bức ảnh, dòng chữ đi kèm ảnh, và khả năng suy luận mượn từ một mô hình ngôn ngữ. Với mỗi nguồn, mục nói rõ nó chống nghiệm tằm thường bằng cơ chế nào và hỏng ở đâu. Đọc xong, trước một bài toán có vài trăm ảnh nhãn, bạn quyết được ngay ba việc: đóng băng hay tinh chỉnh, chọn backbone tự giám sát nào, và công thức augmentation nào sẽ phá hỏng đúng tín hiệu bạn cần. Đoán mò ba việc đó thì mỗi cái tốn vài ngày GPU mới biết là sai. Nếu chỉ nhớ một điều: mỗi bất biến bạn ép vào augmentation chính là một loại thông tin bạn cố tình vứt đi, nên hãy liệt kê các bất biến đó trước khi sao chép công thức của bất kỳ paper nào.

KIẾN THỨC CẦN CÓ TRƯỚC

- **nhãn (label) và tín hiệu giám sát** — nhãn là đáp án đúng gắn với một mẫu dữ liệu; tín hiệu giám sát là bất cứ thứ gì đóng vai trò đáp án để tính hàm mất mát. Mọi ý tưởng trong mục này là cách tạo tín hiệu giám sát mà không phải trả tiền cho nhãn.
- **đặc trưng (feature) và embedding** — đặc trưng là vector số mà mạng sinh ra để mô tả một bức ảnh; *embedding* là cách gọi khác khi ta quan tâm tới hình học của không gian đó — ảnh giống nhau thì vector gần nhau.
- **đóng băng (frozen) và linear probe** — đóng băng là giữ nguyên trọng số backbone, không cho gradient cập nhật. *Linear probe* là huấn luyện đúng một tầng tuyến tính trên đặc trưng đóng băng — phép thử rẻ nhất để xem đặc trưng có chứa thông tin cần thiết hay không.
- **fine-tune (tinh chỉnh)** — huấn luyện tiếp một mô hình đã có trọng số, thường với *learning rate* nhỏ hơn nhiều lần để không xóa mất thứ nó đã học.
- **augmentation** — biến đổi ngẫu nhiên áp lên ảnh lúc huấn luyện (cắt, lật, đổi màu, làm mờ) nhằm dạy mô hình rằng những biến đổi đó không làm đổi đáp án. Chính vì thế nó cũng *xóa* thông tin: bất biến nào bạn dạy thì thông tin đó mất.
- **contrastive (tương phản)** — kiểu học kéo các cặp “giống nhau” lại gần và đẩy các cặp “khác nhau” ra xa trong không gian đặc trưng. Cặp giống nhau gọi là *positive*, cặp khác nhau gọi là *negative*.
- **masked modeling** — che một phần đầu vào rồi bắt mô hình khôi phục phần bị che. Vì đáp án chính là dữ liệu gốc, không cần nhãn nào cả.
- **zero-shot** — dùng mô hình cho một lớp hoặc một tác vụ mà nó chưa từng được huấn luyện trên đó, chỉ bằng cách mô tả lớp ấy bằng ngôn ngữ.

A. Đặt vấn đề

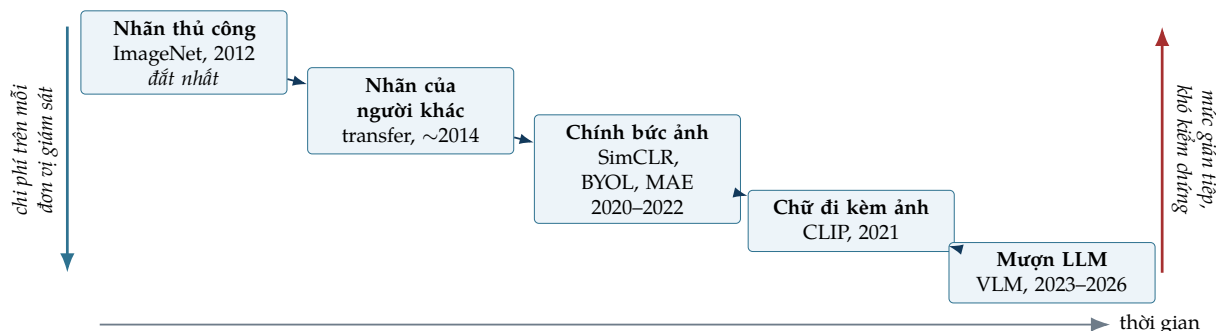
Nhãn đất, dữ liệu không nhãn thì gần như vô hạn. Một bộ ảnh nội soi có nhãn tổn thương cần bác sĩ ngồi vẽ *mask*; một bộ ảnh vệ tinh có nhãn loại đất cần chuyên gia đối chiếu thực địa. Trong khi đó, ổ cứng của cùng bệnh viện hay cùng cơ quan trắc địa chứa hàng triệu ảnh chưa ai chạm vào. Toàn bộ mục này là chuỗi câu trả lời ngày càng táo bạo cho một câu hỏi duy nhất: *tín hiệu giám sát miễn phí có thể lấy từ đâu?*

Câu hỏi nghe như một mẹo tiết kiệm chi phí, nhưng nó thực ra là câu hỏi về bản chất của việc học. Nhãn do người viết ra không phải thứ duy nhất mang thông tin về thế giới. Thông tin còn nằm ở ba chỗ khác. Hai bức ảnh chụp cùng một vật phải cho đặc trưng giống nhau. Phần bị che của một bức ảnh đoán được từ phần còn lại. Và dòng chú thích dưới một bức ảnh trên internet

nói về đúng bức ảnh đó. Mỗi lần ngành khai thác được một nguồn như vậy, chi phí trên mỗi đơn vị giám sát giảm một bậc. Cái giá cũng tăng một bậc: tín hiệu gián tiếp hơn, nhiều hơn, và khó truy ra mô hình đã học gì hơn.

Học xong mục này thì làm được gì mà trước đó không làm được? Bạn nhìn một bài toán có 300 ảnh nhân và quyết được ngay: đóng băng hay tinh chỉnh, chọn *backbone* tự giám sát nào, và công thức *augmentation* nào sẽ phá hỏng chính tín hiệu bạn cần. Đoán mò ba quyết định đó thì mỗi cái tốn vài ngày GPU mới lộ ra là sai.

Hình 1 đặt cả chuỗi lên một trục thời gian trước khi ta đi vào từng bước. Điều đáng nhìn không phải các mốc năm mà là *hai mũi tên ngược chiều nhau* ở hai bên: chi phí đi xuống trong khi mức gián tiếp đi lên. Toàn bộ chương là câu chuyện về việc đánh đổi giữa hai trục ấy.



Hình 1. Bốn nguồn giám sát trên một trục thời gian. Mỗi bước đi xuống một bậc chi phí và đi lên một bậc gián tiếp; không bước nào miễn phí thật sự, chúng chỉ dời hoá đơn từ “thuê người gán nhãn” sang “không biết mô hình đã học gì”.

TRỰC GIÁC

Nghĩ về nó như bài toán trắc địa. Để dựng một bản đồ, bạn không cần toạ độ tuyệt đối của từng cột mốc — một việc đắt như thuê máy đo GNSS cho mỗi điểm. Bạn chỉ cần đủ nhiều *ràng buộc tương đối* giữa các cột mốc (khoảng cách, góc), rồi giải hệ; hình dạng mạng lưới hiện ra chính xác, chỉ còn thiếu một phép biến đổi toàn cục — *gauge* — mà một vài điểm chuẩn rẻ tiền là ghim lại được.

Học tự giám sát làm đúng như vậy trong không gian đặc trưng. Mỗi cặp *augmentation* của cùng một ảnh là một ràng buộc “hai điểm này phải gần nhau”; mỗi cặp ảnh khác nhau là ràng buộc “hai điểm này phải xa nhau”. Giải cả hệ ra được một không gian đặc trưng có hình dạng đúng nhưng chưa gán nhãn — tức là xác định tới một *gauge*. Vài trăm ảnh có nhãn sau đó đóng vai trò các điểm chuẩn: *linear probe* chính là bước ghim *gauge*, và nó rẻ vì nó chỉ phải xác định một phép biến đổi chứ không phải toàn bộ hình dạng.

Mạch chính — bốn nguồn giám sát, theo thứ tự rẻ dần

- Vấn đề** gán nhãn cho bài toán của mình quá đắt, nhưng người khác đã gán nhãn cho bài toán của họ.

Giải pháp **học chuyển giao** (*transfer learning*): lấy trọng số đã học trên ImageNet [1] làm điểm khởi đầu thay vì khởi tạo ngẫu nhiên.

Giả định các tầng đầu của mạng học những thứ chung cho mọi ảnh tự nhiên — cạnh, kết cấu, gradient màu — nên chúng không phụ thuộc vào việc nhãn đầu ra là gì.

Giá phải trả bạn thừa hưởng cả những thiên lệch của nguồn, kể cả những thiên lệch bạn không muốn.

- 2 Vấn đề** ngay cả nhãn của người khác cũng có giới hạn, và miền của bạn có thể chẳng ai gán nhãn.
- Giải pháp** **học tự giám sát** (*self-supervised learning*): tạo bài toán phụ mà đáp án suy ra được từ chính bức ảnh.
- Giả định** cấu trúc thống kê của ảnh đủ giàu để ép mô hình học biểu diễn có nghĩa khi ta buộc nó giải bài toán phụ đó.
- Vấn đề mới** bài toán phụ có nghiệm tầm thường — mô hình xuất ra một hằng số là thoả mọi ràng buộc “giống nhau”. Chống lại sụp biểu diễn (*collapse*: mọi ảnh cho cùng một vector) này là toàn bộ nội dung kỹ thuật của mảng này.
- 3 Vấn đề** ảnh tự nó không nói cho ta biết ngữ nghĩa; nó chỉ nói về tương quan pixel.
- Giải pháp** CLIP và tiền huấn luyện **thị giác-ngôn ngữ** (*vision-language*) [2]: dùng dòng chú thích đi kèm ảnh trên internet làm giám sát.
- Hệ quả** ngôn ngữ trở thành nguồn giám sát ngữ nghĩa rẻ nhất từng có, và nó mang theo một tập nhãn *mở* thay vì 1000 lớp cố định.
- Giá phải trả** dữ liệu web bẩn, thiên lệch, và không kiểm toán được.
- 4 Vấn đề** mô hình vẫn chỉ nhận dạng, chưa suy luận.
- Giải pháp** **mô hình thị giác-ngôn ngữ** (*vision-language model, VLM*) — nối một *vision encoder* vào một LLM đã biết suy luận qua một tầng chiếu hoặc *cross-attention* [3], [4].
- Giả định** biểu diễn thị giác có thể được “phiên dịch” thành thứ mà không gian *token* của LLM hiểu được, và phần suy luận không cần học lại.
- Giá phải trả** đắt, khó kiểm soát, và **ảo giác** (*hallucination*) của phần ngôn ngữ lan sang phần thị giác.
- Hệ quả** tích lũy của cả bốn bước: **mô hình nền tảng** (*foundation model*) trở thành một *thành phần dựng sẵn*, giống thư viện BLAS chứ không giống một mô hình. Nhiều khi dùng đặc trưng đã tiền huấn luyện tốt hơn hẳn tự train, *kể cả trong lĩnh vực hẹp* như ảnh y tế hay ảnh vệ tinh. Điều này trái trực giác, nhưng đã lặp lại đủ nhiều lần để tin [5].

B. Nguồn 1 — nhãn của người khác (Transfer Learning)

Học chuyển giao là kỹ thuật cũ nhất và vẫn là kỹ thuật đáng thử đầu tiên. Điều đáng nói không phải “nó hoạt động”, mà là *quyết định thực tế gói gọn trong một bảng bốn ô*. Học thuộc bảng này tiết kiệm rất nhiều thí nghiệm vô ích.

Bảng 1. Bốn ô quyết định của học chuyển giao. Trục dọc là khoảng cách miền, trục ngang là lượng dữ liệu có nhãn của bạn.

	Dữ liệu ít	Dữ liệu nhiều
Miền giống nguồn	<i>Linear probe</i> trên đặc trưng đóng băng	<i>Fine-tune</i> toàn bộ
Miền khác nguồn	<i>Fine-tune</i> vài tầng cuối, <i>augmentation</i> mạnh	<i>Fine-tune</i> toàn bộ; cân nhắc train từ đầu

Nói gọn thì logic bên dưới Bảng 1 chỉ có một câu: *số tham số bạn cho phép thay đổi phải tương xứng với lượng thông tin mới mà dữ liệu của bạn mang tới*. Đây chính là trục đánh đổi giả định–dữ liệu đã gặp ở B/04-nen-tang-hoc-tu-du-lieu, lần này xuất hiện dưới dạng “mở bao nhiêu tầng”. Ba hệ quả cụ thể:

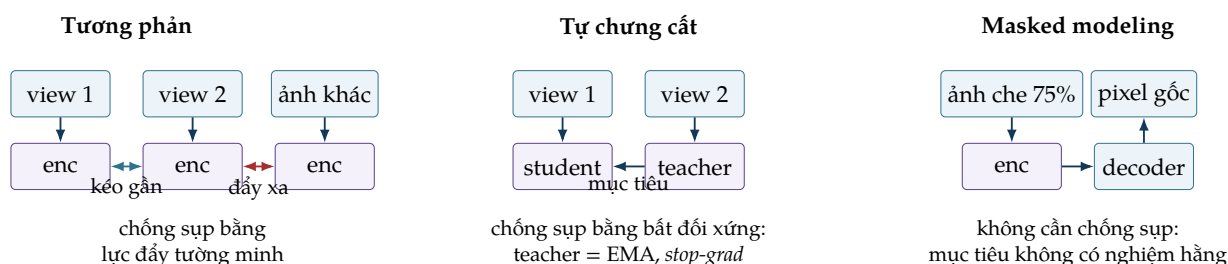
↪ cùng nguyên lý với B/04 — đây chính là trục đánh đổi giả định–dữ liệu, lần này xuất hiện dưới dạng “mở bao nhiêu tầng thì vừa với lượng nhân đang có.

- **Dữ liệu ít** ⇒ **mở ít**. Mở toàn bộ mạng là mời gọi *overfitting* và **quên thảm khốc** (*catastrophic forgetting*) — mạng quên đặc trưng tổng quát để chạy theo vài trăm mẫu.
- **Miền xa** ⇒ **thay tầng cuối**. Các tầng cuối chứa thông tin chuyên biệt của nguồn nên phải thay; các tầng đầu vẫn dùng được vì cạnh vẫn là cạnh.
- **Cả hai điều kiện thuận** ⇒ **tiền huấn luyện chỉ còn là khởi tạo tốt**. Khi bạn có nhiều dữ liệu và miền xa, lợi ích của tiền huấn luyện thu hẹp dần và đôi khi biến mất.

Một chi tiết dễ bị bỏ qua: chất lượng mô hình nguồn trên ImageNet có tương quan với chất lượng chuyển giao, nhưng tương quan đó *yếu đi rõ rệt* khi miền đích xa [6]. Nói cách khác, đừng chọn *backbone* chỉ vì nó đứng cao trên bảng ImageNet; với ảnh nội soi hay ảnh radar, thứ hạng đó gần như không mang thông tin.

C. Nguồn 2 – chính bức ảnh (Self-Supervised Learning)

Học tự giám sát tách thành ba triết lý khác nhau về cùng một câu hỏi: làm sao tạo tín hiệu từ hư không mà không rơi vào nghiệm tầm thường. Hình 2 cho thấy ba *cấu trúc đồ thị tính toán* khác nhau, và chính sự khác nhau về cấu trúc — chứ không phải về **hàm mất mát** (*loss function*) — mới là điều đáng nhớ.



Hình 2. Ba triết lý tự giám sát khác nhau ở *cấu trúc đồ thị*, không ở hàm mất mát. Tương phản chống sụp bằng lực đẩy từ *negative*; tự chưng cất chống sụp bằng bất đối xứng kiến trúc; *masked modeling* không cần chống sụp vì khôi phục pixel không có nghiệm hằng số.

Bảng 2. Ba họ tự giám sát. Cột cuối là cột đáng tiền nhất.

Triết lý	Cơ chế chống sụp	Giả định ngầm	Hỏng khi nào
Tương phản (<i>contrastive</i>) (SimCLR, MoCo)	Lực đẩy tường mình từ rất nhiều <i>negative</i>	Hai ảnh khác nhau trong <i>batch</i> thì thực sự khác lớp	Dữ liệu ít lớp hoặc lặp nhiều: <i>negative</i> hoá ra là <i>positive</i> ; <i>batch</i> nhỏ làm tín hiệu yếu
Tự chưng cất (<i>self-distillation</i>) (BYOL, DINO)	Bất đối xứng kiến trúc: teacher EMA (trung bình trượt, cập nhật chậm theo student), <i>stop-gradient</i> , <i>centering</i>	Điểm cố định của động lực học huấn luyện không phải hằng số	Cơ chế mong manh: đổi tốc độ EMA hay bỏ <i>predictor</i> là sụp; rất nhạy siêu tham số
Masked modeling (MAE)	Không cần: khôi phục pixel không có nghiệm tầm thường	Ngữ nghĩa nằm trong khả năng nội suy phân bị che	Đặc trưng thô, <i>linear probe</i> kém; phải <i>fine-tune</i> mới mạnh — tức đắt hơn ở phía sau

Bảng 2 nói rằng ba họ không cạnh tranh trên cùng một trục. Tương phản [7], [8] và tự chưng cất [9], [10] cho đặc trưng dùng được ngay với *linear probe*, hợp khi bạn định đóng băng *backbone*.

Masked modeling [11] cho đặc trưng cần tinh chỉnh, hợp khi bạn có đủ dữ liệu đích và muốn tận dụng khả năng mở rộng của ViT.

Ví dụ nhỏ nhất: nhiệt độ quyết định gradient nằm ở đâu

Lấy một *batch* bốn ảnh. Đặc trưng đã chuẩn hoá, **độ tương đồng cosine** (*cosine similarity*) của cặp dương là 0,9 và của ba cặp âm đều là 0,1. Hàm InfoNCE [12] cho một mẫu là

$$L = -\log \frac{e^{s^+/\tau}}{e^{s^+/\tau} + \sum_j e^{s_j^-/\tau}}.$$

Đọc bằng lời: đây là cross-entropy của một bài toán phân loại mà “lớp đúng” là bạn đồng hành của chính mình, còn τ quyết định phân phối đó nhọn tới mức nào. Thay số cho ba giá trị **nhiệt độ** (*temperature*):

- $\tau = 1$: $L = -\log(2,460/5,775) = 0,853$
- $\tau = 0,5$: $L = -\log(6,050/9,714) = 0,474$
- $\tau = 0,1$: $L = -\log(8103,1/8111,2) = 0,0010$

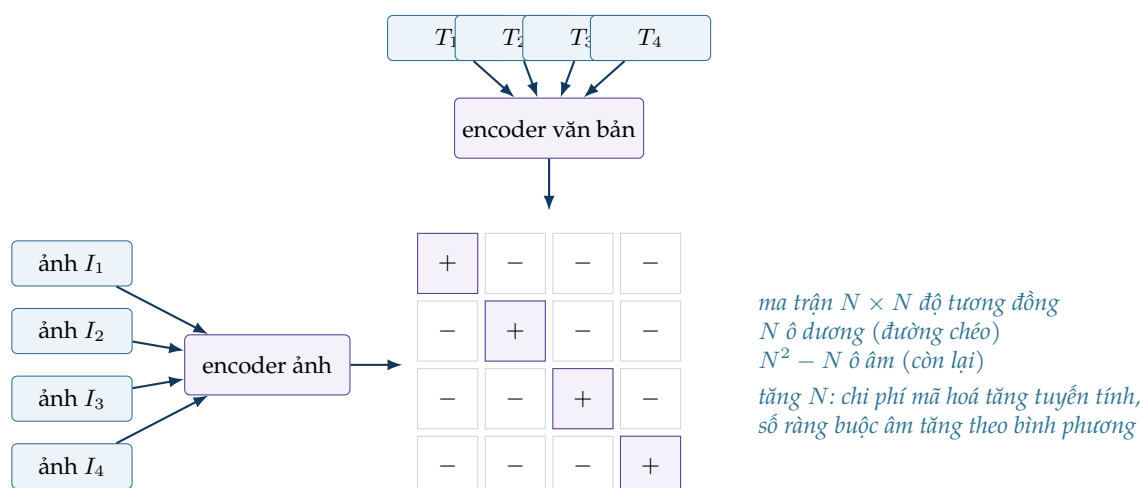
Ba con số nói rõ một điều mà công thức tổng quát giấu đi. Ở nhiệt độ thấp, một cặp đã tách được ở mức tầm thường (0,9 so với 0,1) đóng góp gần như *không* gradient; toàn bộ gradient dồn vào những *negative* có độ tương đồng gần với *positive*. Nhiệt độ vì thế không phải siêu tham số trang trí — nó là núm vặn quyết định hàm mất mát chú ý tới *negative* khó tới mức nào. Vặn quá thấp thì mô hình chỉ học từ vài cặp khó nhất và trở nên nhạy với nhiễu ngẫu nhiên; vặn quá cao thì mọi cặp đóng góp như nhau và biểu diễn không tách được các lớp gần nhau. Đây đúng là cơ chế đã gặp ở B/09-thiet-ke-ham-mat-mat dưới tên *hard negative mining*, chỉ khác là ở đây nó được điều khiển bằng một số thực thay vì bằng một bộ lọc.

↔ cùng nguyên lý với B/09 — nhiệt độ dồn gradient về các cặp khó chính là hard negative mining, chỉ khác là ở đây nó được điều khiển bằng một số thực thay vì bằng một bộ lọc.

D. Nguồn 3 — văn bản đi kèm ảnh (Vision-Language Pretraining)

CLIP giữ nguyên ý tưởng tương phản nhưng đổi định nghĩa cặp dương: thay vì hai *augmentation* của cùng một ảnh, cặp dương là (ảnh, dòng chú thích của chính nó), và cặp âm là mọi tổ hợp chéo còn lại trong *batch*. Với *batch* kích thước N , một lượt tính cho ma trận $N \times N$ độ tương đồng và $2N$ bài toán phân loại N lớp — đó là lý do CLIP thêm *batch* lớn tới mức bất thường.

Hình 3 cho thấy vì sao một *batch* lớn lại quan trọng đến thế: mỗi lượt chỉ có N ô dương trên đường chéo nhưng $N^2 - N$ ô âm, nên tăng N làm số ràng buộc âm tăng theo bình phương trong khi chi phí mã hoá chỉ tăng tuyến tính.



Hình 3. CLIP đối định nghĩa cặp dương: thay vì hai *augmentation* của cùng một ảnh, cặp dương là (ảnh, chú thích của chính nó). Một lượt cho N bài toán phân loại theo hàng và N bài theo cột trên cùng một ma trận — đó là toàn bộ lý do CLIP thêm *batch* lớn bất thường.

Nhận thức then chốt không nằm ở kiến trúc mà ở tính chất của tín hiệu:

- **Nhãn ImageNet** là một hàm từ ảnh vào một tập 1000 phần tử *cố định*.
- **Chú thích văn bản** là một hàm vào một không gian ngữ nghĩa *mở*.

Hệ quả trực tiếp là phân loại *zero-shot*: muốn thêm một lớp mới, bạn viết thêm một câu chứ không gán thêm nhãn. Hệ quả xa hơn là toàn bộ dòng **phát hiện từ vựng mở** (*open-vocabulary detection*) và phân vùng ở chương 14. Chúng chỉ ghép bộ đề xuất vùng với không gian *embedding* văn bản này, nên mọi điểm yếu của CLIP di truyền nguyên vẹn sang đó.

↪ **hệ quả của C/14** — phát hiện từ vựng mở là bộ đề xuất vùng cắm vào chính không gian embedding này, nên nó thừa hưởng nguyên vẹn điểm yếu của CLIP về quan hệ không gian và đếm.

Cái giá thì cụ thể và đã được đo nhiều lần. Mô hình học từ chú thích web hành xử gần với một túi từ hơn là một bộ phân tích cú pháp: nó phân biệt tốt “có con chó” với “có con mèo”, nhưng yếu ở quan hệ không gian (“con mèo bên trái con chó”), ở đếm số lượng, và ở phủ định. Kết quả *zero-shot* còn nhạy với cách viết *prompt* tới mức khó chịu. Chênh lệch giữa “a photo of a {}” và “{}” có thể tới vài điểm phần trăm. Hãy đọc con số đó như dấu hiệu mô hình chưa ổn định, không phải như một mẹo tối ưu.

E. Nguồn 4 — ghép thị giác vào một LLM (Multimodal LLM)

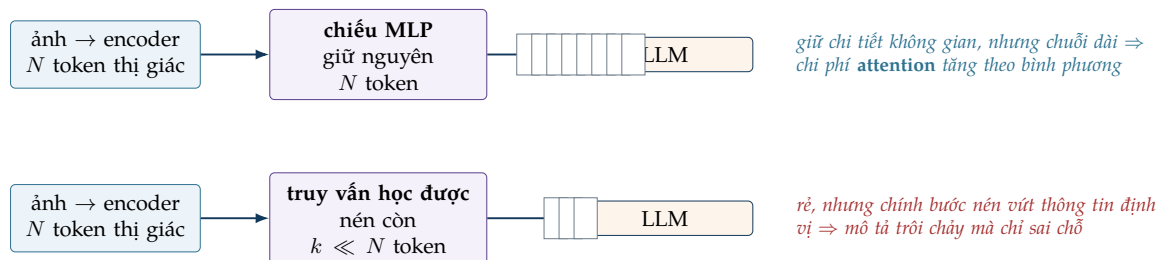
Bước tiếp theo hỏi: nếu đã có một mô hình ngôn ngữ biết suy luận, tại sao phải dạy lại phần suy luận cho thị giác? Kiến trúc VLM vì thế rất đơn giản về ý tưởng và gồm đúng ba khối:

1. **Vision encoder** — thường là CLIP hoặc SigLIP — biến ảnh thành một chuỗi *token* thị giác.
2. **Khối kết nối** — một MLP chiếu, hoặc một tập truy vấn học được kiểu Q-Former [3] — đưa chuỗi đó về cùng không gian với *token* văn bản.
3. **LLM** — xử lý chuỗi hỗn hợp hệt như xử lý văn bản thuần [4].

Trọng tâm thiết kế nằm ở khối thứ hai, và nó là một cuộc đánh đổi kinh điển. Chiếu MLP thẳng giữ nguyên số *token* thị giác nên giữ được chi tiết không gian, nhưng làm chuỗi dài ra và chi phí *attention* tăng theo bình phương. Nén xuống một số ít *token* truy vấn thì rẻ, nhưng chính bước nén đó vứt đi thông tin định vị — và đó là lý do nhiều VLM mô tả ảnh trôi chảy nhưng chỉ sai vị trí. Cái được là VQA, *grounding*, *captioning* và điều khiển giao diện bằng một mô hình duy

nhất; cái mất là chi phí, khả năng kiểm soát, và một dạng lỗi mới: mô hình bịa ra chi tiết không có trong ảnh với đúng giọng tự tin mà phần ngôn ngữ đã học được.

Hình 4 đặt hai biến thể của khối kết nối cạnh nhau, vì chính chỗ này quyết định VLM của bạn định vị được hay không.



Hình 4. Hai cách nối bộ mã hoá thị giác vào một LLM. Số ô nhỏ giữa khối kết nối và LLM là số **token** thị giác thực sự đi vào chuỗi; đó là đại lượng duy nhất quyết định cả chi phí lẫn khả năng định vị, nên đây là lựa chọn thiết kế quan trọng nhất của một VLM.

F. Kết quả tích lũy – mô hình nền tảng như một thành phần

Điều đáng ngạc nhiên nhất của cả mục không phải là mô hình nền tảng mạnh, mà là nó mạnh ở *những nơi lẽ ra nó không nên mạnh*. Lấy đặc trưng DINOv2 hoặc DINOv3 đóng băng rồi gắn thêm một đầu tuyến tính, bạn thường thắng một mạng train từ đầu ngay trên dữ liệu chuyên ngành [5]. Điều đó đúng với ảnh nội soi, ảnh vệ tinh và ảnh vi thể, ở mức vài trăm tới vài nghìn nhãn. Trực giác thông thường nói dữ liệu trong miền phải thắng dữ liệu ngoài miền; thực nghiệm nói vài trăm mẫu không đủ để học lại những thống kê ảnh cơ bản mà 140 triệu ảnh đã học họ.

Điều này thay đổi vị trí của bạn trong hệ thống: bạn không còn là người huấn luyện mô hình mà là người *tích hợp* nó — giống hệt việc dùng một solver phi tuyến có sẵn thay vì tự viết Levenberg–Marquardt. Và cũng giống hệt như vậy, trách nhiệm chuyển từ “mô hình có đúng không” sang “tôi có biết mô hình này giả định gì và hỏng ở đâu không”.

G. Giới hạn và trường hợp hỏng

- **“Mở trọng số” không đồng nghĩa với “mở”.** Một số *backbone* tự giám sát mạnh nhất phát hành kèm giấy phép riêng hạn chế dùng thương mại, và dữ liệu huấn luyện thì hầu như không bao giờ được công bố. Với sản phẩm, đây là ràng buộc cứng cần kiểm tra *trước* khi chạy thí nghiệm.
- **Chuyển giao thất bại có hệ thống khi vật lý tạo ảnh khác nguồn** — ảnh nhiệt, radar, ảnh sự kiện, đa phổ. Ở đó thống kê tần số, phân bố nhiễu và cả số kênh đều khác, nên giả định “cạnh vẫn là cạnh” không còn đúng (xem A/02-vat-ly-tao-anh/04-cam-bien-khac-va-domain-gap).
- **Tự giám sát không miễn nhiễm với thiên lệch dữ liệu; nó chỉ làm thiên lệch khó nhìn hơn**, vì không còn bảng phân bố nhãn để kiểm tra. Nếu 90% ảnh không nhãn của bạn chụp ban ngày, biểu diễn học ra sẽ tốt ban ngày và không ai cảnh báo bạn.
- **Đánh giá bằng *linear probe* che giấu rất nhiều.** Một biểu diễn có thể tuyến tính phân tách tốt các lớp phổ biến mà vẫn vô dụng cho định vị chính xác — đúng loại thất bại quan trọng nhất nếu đầu ra của bạn là hộp, *mask* hay tương ứng điểm.

CẠM BẤY

Bất biến mà bạn ép vào *augmentation* chính là thông tin bạn cố tình vớt đi. *Color jitter* mạnh dạy mô hình rằng màu không quan trọng — một thảm họa nếu tác vụ đích là phát

hiện gỉ sét, phân loại nhuộm mô bệnh học, hay đọc đèn tín hiệu. *Random resized crop* dạy mô hình rằng tỉ lệ không quan trọng — một thảm họa nếu bạn cần ước lượng kích thước thật. Trước khi sao chép một công thức *augmentation* từ paper tự giám sát, hãy liệt kê các bất biến nó áp đặt và đối chiếu từng cái với thứ tác vụ của bạn cần *phân biệt*.

CẦN NHỚ

Mỗi bước trong chuỗi này đòi *nhân vất vả* và *chính xác lấy tín hiệu rẻ và gián tiếp*. Nó gần như luôn có lỗi, nhưng cái bạn mất là khả năng biết mô hình đã học gì — và bạn chỉ phát hiện ra điều đó ở đúng những trường hợp biên mà bạn quan tâm nhất. Hệ quả hành động: với mọi mô hình nền tảng, hãy tự thu một tập đánh giá nhỏ trên đúng phân phối triển khai trước khi tin bất kỳ con số nào. [T2]

H. Kết nối

Mục này là tiền đề trực tiếp của C/14-bai-toan-cv-loi: phát hiện từ vụng mở chỉ là bộ đề xuất vùng cấm vào không gian *embedding* CLIP. Nó đối lập với B/06-chat-luong-du-lieu. Chương kia lập luận rằng chất lượng nhãn quyết định trần hiệu năng; mục này lập luận rằng phần lớn tín hiệu lấy được mà không cần nhãn. Cả hai đều đúng, và ranh giới nằm ở chỗ bạn cần *ngữ nghĩa* hay cần *biểu diễn*. Về phía sau, C/18/02-kinh-te-hoc-pretraining trả lời câu hỏi định lượng: đầu tư compute vào đâu, và khi nào tinh chỉnh đáng giá hơn đóng băng.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao BYOL không sụp về nghiệm hằng số dù không có một *negative* nào, và bỏ thành phần nào thì nó sụp?
2. Với 500 ảnh nội soi, *linear probe* trên DINOv3 hay *fine-tune* toàn bộ ResNet ImageNet sẽ thắng, và lập luận đó dựa trên đại lượng nào — “số tham số được phép thay đổi so với lượng thông tin mới” nói gì ở đây?
3. Hạ nhiệt độ τ trong InfoNCE làm gradient dịch chuyển đi đâu, và tác dụng phụ nào xuất hiện khi tập dữ liệu có nhiều ảnh gần trùng nhau?
4. MAE cho *linear probe* kém hơn DINO nhưng tinh chỉnh lại tốt hơn. Điều đó nói gì về loại thông tin mà hai mục tiêu huấn luyện giữ lại?
5. Vì sao “mở trọng số” không đồng nghĩa với “mở”, và điều đó ảnh hưởng gì tới lựa chọn *backbone* cho một sản phẩm thương mại?
6. Tác vụ nào sẽ bị công thức *augmentation* chuẩn của SimCLR làm hỏng biểu diễn, và bất biến nào bị ép sai ở đó?

Nguồn

- [1] J. Deng, W. Dong, R. Socher, et al., “ImageNet: A Large-Scale Hierarchical Image Database,” in *CVPR*, 2009.
- [2] A. Radford, J. W. Kim, C. Hallacy, et al., “Learning Transferable Visual Models from Natural Language Supervision,” in *ICML*, 2021.
- [3] J. Li, D. Li, S. Savarese, et al., “BLIP-2: Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models,” in *ICML*, 2023.
- [4] H. Liu, C. Li, Q. Wu, et al., “Visual Instruction Tuning,” in *NeurIPS*, 2023.
- [5] M. Oquab, T. Darcet, T. Moutakanni, et al., “DINOv2: Learning Robust Visual Features Without Supervision,” *TMLR*, 2024.
- [6] S. Kornblith, J. Shlens, and Q. V. Le, “Do Better ImageNet Models Transfer Better?” In *CVPR*, 2019.
- [7] T. Chen, S. Kornblith, M. Norouzi, et al., “A Simple Framework for Contrastive Learning of Visual Representations,” in *ICML*, 2020.
- [8] K. He, H. Fan, Y. Wu, et al., “Momentum Contrast for Unsupervised Visual Representation Learning,” in *CVPR*, 2020.
- [9] J.-B. Grill, F. Strub, F. Altché, et al., “Bootstrap Your Own Latent: A New Approach to Self-Supervised Learning,” in *NeurIPS*, 2020.
- [10] M. Caron, H. Touvron, I. Misra, et al., “Emerging Properties in Self-Supervised Vision Transformers,” in *ICCV*, 2021.
- [11] K. He, X. Chen, S. Xie, et al., “Masked Autoencoders Are Scalable Vision Learners,” in *CVPR*, 2022.
- [12] A. van den Oord, Y. Li, and O. Vinyals, “Representation Learning with Contrastive Predictive Coding,” in *arXiv:1807.03748*, 2018.